Deprecated Threading Methods

```
import threading


class MultithreadingDemo(threading.Thread):

    def __init__(self):
        threading.Thread.__init__(self)
        self.__name = None

    def get_name(self):
        return self.__name

    def set_name(self, name):
        self.__name = name

    def run(self):
        s = threading.currentThread().getName() + " is running"
        self.set_name(s)
        print(self.__name)


class Multithread:

    @staticmethod
    def main():
        n = int(input("Enter the number of threads to spawn: "))
        for i in range(n):
            obj = MultithreadingDemo()
            obj.start()


if __name__ == "__main__":
    Multithread.main()
```

The function "currentThread" is deprecated
Deprecated since Python 3.10. Use `current_thread()` instead.

You're getting that warning because in **Python 3.10+**,

```
threading.currentThread()
```

is **deprecated** and replaced with:

```
threading.current_thread()
```

✅ Fix in Your Code

Change this line inside `run()`:

```python
s = threading.currentThread().getName() + " is running"
```

to:

```python
s = threading.current_thread().name + " is running"
```

🔥 Even Better (Modern Python Style)

Also, `getName()` is deprecated. You should directly use `.name`.

✔️ Updated Version of Your Code:

```python
import threading


class MultithreadingDemo(threading.Thread):

    def __init__(self):
        super().__init__()
        self.__name = None

    def get_name(self):
        return self.__name

    def set_name(self, name):
        self.__name = name

    def run(self):
```

```
        s = threading.current_thread().name + " is running"
        self.set_name(s)
        print(self.__name)


class Multithread:

    @staticmethod
    def main():
        n = int(input("Enter the number of threads to spawn: "))
        for i in range(n):
            obj = MultithreadingDemo()
            obj.start()


if __name__ == "__main__":
    Multithread.main()
```

🧠 Why It Changed?

Python developers renamed:

- `currentThread()` → `current_thread()`
- `getName()` → `.name`

To follow **PEP 8 naming conventions** (snake_case style).

If you want, I can also show you a **cleaner Pythonic way** of writing multithreading without creating a separate class (using `Thread(target=...)`).

> i am new to threading in python so explain the program first so that there is soem catching point by which i can rememebr the threading in python
> although i have doen threading in java so i know how it works in backend and basic it kust new feature and syntax and implementation in pyhton that i,m not aware poff

Perfect 👌 since you already know **Java threading**, I'll explain this in a way that connects **Java → Python**, so you get strong memory hooks.

🧠 First Big Catching Point

👉 In Python, a thread is just:

> A class that extends `threading.Thread` and overrides `run()`

Exactly like Java.

Java	Python
`extends Thread`	`class MyThread(Thread)`
override `run()`	override `run()`
`start()`	`start()`

So mentally remember:

> **Python threading = Java threading, just snake_case syntax**

📦 Step-by-Step Explanation of Your Program

1️⃣ `import threading`

This is like:

```
Java
import java.lang.Thread;
```

Python keeps threading inside the `threading` module.

2️⃣ Class Definition

```
Python
class MultithreadingDemo(threading.Thread):
```

🔁 Java Equivalent:

```java
class MultithreadingDemo extends Thread
```

So this class **is a thread**.

3️⃣ Constructor

```python
def __init__(self):
    threading.Thread.__init__(self)
    self.__name = None
```

🔁 Java Equivalent:

```java
MultithreadingDemo() {
    super();
}
```

Important:

In Python you must manually call parent constructor.

Cleaner way:

```python
super().__init__()
```

4️⃣ The `run()` Method (🔥 Most Important)

```python
def run(self):
```

This is EXACTLY like Java.

⚡ Very Important Concept

When you call:

```python
obj.start()
```

Python internally calls:

```python
obj.run()
```

Just like Java.

5️⃣ Inside `run()`

```python
s = threading.current_thread().name + " is running"
print(self.__name)
```

What is happening?

- `current_thread()` → returns the currently executing thread object
- `.name` → gives default name like:
  - Thread-1
  - Thread-2
  - Thread-3

So output looks like:

```
Thread-1 is running
Thread-2 is running
Thread-3 is running
```

6️⃣ The Main Part

```python
for i in range(n):
    obj = MultithreadingDemo()
    obj.start()
```

Each iteration:

1. Create new thread object
2. Call `start()`
3. Thread scheduler runs `run()` in parallel

🔥 What Is Actually Happening Internally?

When you do:

```python
obj.start()
```

Python:

1. Allocates OS-level thread
2. Registers thread in interpreter
3. Calls `run()` method
4. Runs it concurrently

⚠️ VERY IMPORTANT (Since You Know Java)

🧠 Catching Point About Python Threading

Python has something called:

🧵 GIL (Global Interpreter Lock)

This is the biggest difference from Java.

Java:

True parallelism for CPU tasks.

Python:

Due to GIL:

- Only **one thread executes Python bytecode at a time**
- Threads are good for:
  - I/O operations
  - Network calls
  - File handling

- Waiting tasks

Not good for heavy CPU tasks.

🔑 Memory Hooks For You

Since you are from Java background, remember this:

🎯 Hook 1:

> Python threading syntax is lighter but concept same as Java

---

🎯 Hook 2:

> `start()` calls `run()` internally (Same as Java)

🎯 Hook 3:

> Python threads share memory automatically (no need for special setup)

🎯 Hook 4:

> For CPU heavy work → use multiprocessing, not threading

🆚 Python Threading vs Java Threading

Feature	Java	Python
True parallel CPU	✅ Yes	❌ No (GIL)
I/O threading	✅ Yes	✅ Yes
Need synchronized	Yes	Yes (Lock)
Extend Thread	Yes	Yes

Feature	Java	Python
Runnable interface	Yes	Use `target=`

💡 Cleaner Python Way (Like Runnable in Java)

Instead of extending Thread, Python often uses:

```python
import threading

def task():
    print(threading.current_thread().name, "is running")

t1 = threading.Thread(target=task)
t1.start()
```

🔁 Java Equivalent:

```java
new Thread(() -> {
    System.out.println(Thread.currentThread().getName());
}).start();
```

🎓 Final Mental Model

Think of Python threading as:

> Java Threading Lite Version + GIL Limitation

If you want next level understanding, I can explain:

- How scheduler works in Python
- What exactly GIL locks
- When to use threading vs multiprocessing
- Real interview style tricky questions

Tell me what level depth you want 🚀

super().__init__()

when we call with super dont we give self and what is thsi self explain in conetxt in jav if this type of feature is in java

Excellent question 👌

Since you know Java, I'll connect `self` **in Python** with `this` **in Java** so it clicks instantly.

🔹 1️⃣ What is `self` in Python?

In Python:

```python
def method(self):
```

`self` = reference to the **current object**

🔁 Java Equivalent:

```java
void method() {
    // here "this" is automatically available
}
```

In Java:

- `this` is **implicitly available**
- You don't pass it manually

In Python:

- `self` must be written explicitly as the first parameter
- But Python passes it automatically when you call the method

---

🔹 2️⃣ Why Don't We Pass `self` in `super().__init__()`?

Look at this:

```python
```

```
super().__init__()
```

You might think:

> Shouldn't we write `super().__init__(self)` ?

Answer: ❌ No.

Because:

- When you call a method on an object (`something.method()`),
- Python automatically passes the current object reference.

🧠 What Actually Happens Internally

When you write:

```python
super().__init__()
```

Python internally does something like:

```
Thread.__init__(self)
```

So `self` is already passed automatically.

🔁 Java Equivalent of `super()`

Python:

```python
super().__init__()
```

Java:

```java
super();
```

Both call the parent constructor.

🔥 Important Concept: Why Does Python Need `self` At All?

Because in Python:

Functions inside a class are just normal functions.

They only become "methods" when called from an object.

Example:

```python
class A:
    def show(self):
        print("Hello")
```

Internally, Python treats this like:

```python
A.show(obj)
```

So:

```python
obj.show()
```

Is actually:

```
A.show(obj)
```

That `obj` becomes `self`.

🔹 Java vs Python Object Reference

Concept	Java	Python
Current object reference	`this`	`self`
Passed automatically?	✅ Yes	❌ No (must declare in method)

Concept	Java	Python
Written explicitly?	❌ No	✅ Yes

🔥 Now Let's Connect to Your Thread Code

Your constructor:

```python
def __init__(self):
    super().__init__()
```

Here:

1. `self` = current `MultithreadingDemo` object
2. `super()` = gives access to parent class (`Thread`)
3. `super().__init__()` calls:

```
Thread.__init__(self)
```

Which initializes thread internals.

🧠 Memory Hook for You (Java Brain Version)

Think:

> `self` in Python = `this` in Java
> But Python forces you to write it explicitly.

🔹 Advanced Insight (Very Important)

In older Python versions, people wrote:

```python
threading.Thread.__init__(self)
```

This manually passes `self`.

But modern Python recommends:

```python
super().__init__()
```

Cleaner and safer (especially for multiple inheritance).

🎯 Final Understanding in One Line

When you write:

```python
super().__init__()
```

It is EXACTLY like writing in Java:

```java
super();
```

And `self` is automatically handled by Python's method calling mechanism.

If you want, next I can explain:

- How Python actually binds methods to objects internally
- What `super()` really returns
- Why Python supports multiple inheritance better than Java
- Or how GIL interacts with threads at object level

Tell me how deep you want to go 😄

explain gli and what is this Runnable interface
Yes
Use target=

Perfect 😄

Now we'll connect three things clearly:

1. 🔒 **GIL (Global Interpreter Lock)**

2. 🧩 **Runnable Interface (Java concept)**
3. 🎯 **Python `target=` (Runnable equivalent)**

Since you're from Java, I'll map everything Java → Python.

🔒 1️⃣ What is GIL?

📌 Full Form:

Global Interpreter Lock

Used in:

👉 **Python**

🧠 Simple Meaning

> Only ONE thread can execute Python bytecode at a time.

Even if you create 10 threads,

only one runs Python code at a time.

🏗️ Why Does Python Have GIL?

Because:

- Python uses **reference counting** for memory management.
- To avoid memory corruption,
- It locks the interpreter.

So:

```
Thread 1 running Python code
Thread 2 waits
Thread 3 waits
```

They take turns.

🔥 Important Difference From Java

Feature	Java	Python
True CPU parallelism	✅ Yes	❌ No (because of GIL)
I/O parallelism	✅ Yes	✅ Yes

🧠 When Does Python Threading Make Sense?

✅ Good for:

- Network calls
- File I/O
- Database operations
- Waiting tasks

❌ Not good for:

- Heavy CPU calculations
- Matrix multiplication
- Big loops

For CPU work → use **multiprocessing**

🧩 2️⃣ What is Runnable Interface? (Java)

In:

👉 **Java**

You can create threads in two ways:

Method 1: Extend Thread

```java
class MyThread extends Thread {
    public void run() {
        System.out.println("Running");
    }
}
```

Method 2: Implement Runnable (Better Way)

```java
class MyTask implements Runnable {
    public void run() {
        System.out.println("Running");
    }
}

Thread t = new Thread(new MyTask());
t.start();
```

🎯 Why Runnable is Better?

Because:

- Java doesn't support multiple inheritance.
- If you extend Thread, you cannot extend another class.
- Runnable gives flexibility.

🚀 3️⃣ Python Equivalent of Runnable → `target=`

Python does NOT have Runnable interface.

Instead, it uses:

```python
threading.Thread(target=function_name)
```

This is EXACTLY like Runnable.

🔁 Java Runnable vs Python target=

Java:

```java
Thread t = new Thread(() -> {
    System.out.println("Running");
});
t.start();
```

Python:

```python
import threading

def task():
    print("Running")

t = threading.Thread(target=task)
t.start()
```

🧠 Memory Hook

Think:

> `target=` in Python = Runnable in Java

You pass the function that should run inside the thread.

🔥 Why `target=` is More Pythonic

Instead of:

```python
class MyThread(threading.Thread):
    def run(self):
        print("Running")
```

You can just do:

```python
t = threading.Thread(target=task)
```

Cleaner. Shorter. Preferred.

🧠 Visual Comparison

Concept	Java	Python
Extend Thread	`extends Thread`	`class A(Thread)`
Runnable	`implements Runnable`	`target=function`
Start thread	`start()`	`start()`
run() execution	internal	internal

🔥 Final Important Concept

Because of GIL:

Even if you write:

```python
for i in range(5):
    threading.Thread(target=task).start()
```

They are NOT truly parallel for CPU tasks.

They time-slice.

🎯 Clean Modern Python Threading Example

```python
import threading

def task():
    print(threading.current_thread().name, "is running")

for i in range(3):
    t = threading.Thread(target=task)
    t.start()
```

🧠 Final Mental Model For You

Think:

> Java = True multithreading
> Python = Concurrency for I/O + GIL limitation
> Runnable = target=

If you want next level clarity, I can explain:

- Why multiprocessing bypasses GIL
- How context switching works
- Real interview traps about GIL
- Difference between concurrency vs parallelism

Tell me what depth you want 😄

True CPU parallelism
I/O parallelism

what is the difference between them

Very important question 👌
This is where most people get confused.

Let's build this clearly using your **Java background**.

🔥 First: Two Different Problems

1️⃣ CPU-bound work

Work that keeps the **processor busy**

Examples:

- Sorting 10 million numbers
- Matrix multiplication
- Cryptography
- Heavy DSA computations

👉 The CPU is doing calculations continuously.

2️⃣ I/O-bound work

Work that spends most time **waiting**

Examples:

- Calling an API
- Reading a file
- Querying a database
- Downloading data

👉 The CPU is mostly idle waiting for response.

🧠 Now the Main Difference

🔵 True CPU Parallelism

> Multiple threads execute CPU instructions at the same time on different cores.

If you have:

- 4 cores
- 4 threads
- CPU-heavy task

All 4 cores work simultaneously.

Example in:

👉 Java

Java threads can run in true parallel on multi-core CPUs.

So total time reduces significantly.

🟢 I/O Parallelism (Concurrency)

> While one thread waits for I/O, another thread runs.

Example:

Thread 1:

```
Call API → wait 2 seconds
```

During those 2 seconds:

Thread 2 runs.

CPU is not calculating.
It is just switching tasks.

🎯 Timeline Comparison

🧮 CPU Parallelism (True Parallel)

On 4 cores:

```
Core1: Task A
Core2: Task B
Core3: Task C
Core4: Task D
```

All running SAME TIME.

🌐 I/O Parallelism (Waiting-Based)

On 1 core:

```
Task A → waiting for network
Task B → runs
Task B → waiting for disk
Task C → runs
```

Only ONE executes at a time.
But system feels fast because we don't waste waiting time.

🚨 Now Important for Python

In:

👉 Python

Because of **GIL**:

❌ CPU parallelism does NOT happen with threads
✅ I/O parallelism works perfectly

🔥 Simple Real-Life Analogy

CPU Parallelism = 4 chefs cooking 4 dishes simultaneously

I/O Parallelism = 1 chef:

- Put rice on stove
- While rice cooks, chop vegetables
- While vegetables cook, wash dishes

He is not doing 3 things at once,
he is switching while waiting.

🧠 Technical Difference

Feature	CPU Parallelism	I/O Parallelism
Uses multiple cores	✅ Yes	❌ Not required
True simultaneous execution	✅ Yes	❌ No
Best for	Heavy computation	Network, file, DB
Python threading	❌ Limited (GIL)	✅ Good

⚡ Quick Test to Understand

CPU-bound example (bad for Python threads):

```python
for i in range(10**8):
    x = i*i
```

Threads won’t speed this up in Python.

I/O-bound example (good for Python threads):

```python
requests.get("https://example.com")
```

While waiting for response,

other threads can run.

🧠 Final Memory Hook

> CPU Parallelism = Multiple brains thinking simultaneously
> I/O Parallelism = One brain multitasking while waiting

If you want, next I can explain:

- Why multiprocessing solves CPU issue in Python
- How async/await differs from threading
- Or give a diagram showing how GIL blocks CPU threads

What do you want next? 🚀